

Prompt: Design an Enterprise AI Knowledge Platform for Insurance Property Coverage Analysis

You are an Enterprise Solution Architect specializing in AI, Retrieval-Augmented Generation (RAG), Azure Cloud, Insurance Systems, and Production Software Architecture.

I want you to act as a Principal Architect and create a complete architecture proposal suitable for leadership presentation, architecture review, and engineering implementation.

Do not generate a prototype architecture. Instead, design an enterprise-grade platform following software engineering best practices.

Business Context

The application is for an insurance company.

The end users are:

- Property Adjusters
- Claims Professionals

This application is NOT customer-facing.

The goal is to reduce the manual effort required to perform policy coverage analysis.

Currently adjusters manually:

- Retrieve claim information
- Retrieve policy documents
- Copy policy into AI tools
- Ask coverage questions

The new platform should automate this process.

Functional Workflow

The application should collect the following information from the adjuster:

- Claim Number
- Policy Number
- Coverage Questions
- Observations

- Concerns

Once submitted:

Step 1

Using Claim Number

Call Claims API

Retrieve:

- Loss Date
 - Loss Type
 - Loss Description
-

Step 2

Using Policy Number

Call Policy Administration API

Retrieve:

- Policy PDF
 - Policy Metadata
-

Step 3

Before processing

Check whether this policy has already been indexed.

Search by Policy Number.

Decision:

If indexed

Skip indexing.

If not indexed

Run complete indexing pipeline.

Step 4

Knowledge Indexing Pipeline

Pipeline should include:

- Document Acquisition
- Document Parsing
- Cleaning
- Layout Analysis
- Metadata Extraction
- Hierarchical Chunking
- Embedding Generation
- Validation
- Upload to Azure AI Search

The indexing pipeline must be completely custom.

Do NOT use Azure AI Search Indexers.

Azure AI Search should only be used as a search backend.

Step 5

Run Retrieval

Retrieve policy context.

Construct prompt.

Invoke LLM.

Step 6

Generate comprehensive coverage analysis.

The assistant should provide:

- Policy Holder Information
- Policy Number
- Effective Date
- Expiration Date
- Policy Deductible
- Coverages
- Additional Coverages
- Additional Deductibles
- Aggregate Limits

- Aggregate Terms
- Exclusions
- Endorsements
- Mortgagees
- Loss Payees
- Lienholders
- Multiple Locations
- Coverage by Location
- Address by Location
- Page References
- Supporting Policy Language

General Liability sections must NOT be analyzed.

In addition to the structured report, the assistant should answer adjuster-specific coverage questions.

Every answer should contain page references.

Architecture Decisions Already Made

Use these decisions exactly.

Backend

Python

FastAPI

Frontend

React

Cloud

Microsoft Azure

Document Storage

Azure Blob Storage

Search

Azure AI Search

Reason:

Enterprise Search

Hybrid Search

BM25

Vector Search

Metadata Filters

Azure Ecosystem

Do NOT recommend Pinecone.

However,

Architecture must remain provider-independent.

Design Search Provider abstraction.

Future implementations should support:

- Pinecone
- PageIndex

without changing application logic.

Embeddings

Azure OpenAI

text-embedding-3-small

LLM

Azure OpenAI GPT

Hosting

Azure Container Apps

Secrets

Azure Key Vault

Monitoring

Application Insights

Authentication

Microsoft Entra ID

Retrieval Strategy

Use Hybrid Retrieval.

Pipeline:

BM25

+

Dense Vector Search

↓

Merge Results

↓

Optional Reranker

↓

LLM

Do NOT use Sparse Encoder initially.

Explain why.

Indexing Strategy

Do NOT pre-index every policy.

Use On-Demand Indexing.

Workflow:

Coverage Request

↓

Check Index

↓

Indexed?

↓

YES

Retrieve

↓

NO

Run Indexing Pipeline

↓

Store Index

↓

Retrieve

↓

LLM

Policies remain indexed after first processing.

Future requests should skip indexing.

Important Architecture Requirement

The system must be modular.

Separate services:

- Claims Integration
- Policy Integration
- Document Processing
- Knowledge Transformation
- Embedding Service
- Search Provider
- Retrieval Service
- Coverage Analysis Engine

Do NOT build one monolithic service.

Canonical Document Representation

Before embeddings,

create an internal canonical representation.

Example:

Document

↓

Sections

↓

Hierarchy

↓

Metadata

↓

Chunks

↓

Canonical JSON

↓

Embeddings

↓

Azure AI Search

This allows future migration to:

- Pinecone
- PageIndex
- Other Retrieval Technologies

without reparsing documents.

Future Flexibility

Design interfaces for:

Parser

Embedding Provider

Search Provider

Retrieval Provider

Architecture should support future PageIndex integration.

Deliverables Required

Produce a professional architecture document including:

1. Executive Summary
2. Business Problem
3. Proposed Solution
4. User Workflow

5. High-Level Architecture
6. Technical Architecture
7. Sequence Diagram
8. Component Diagram
9. Deployment Architecture
10. Azure Resource Architecture
11. Data Flow
12. API Flow
13. Indexing Pipeline
14. Retrieval Pipeline
15. Coverage Analysis Pipeline
16. Technology Selection
17. Why each technology was selected
18. Alternatives considered
19. Azure AI Search vs Pinecone comparison
20. Cost Analysis
21. Production Deployment
22. Development Environment
23. CI/CD Pipeline
24. Security
25. Authentication
26. Monitoring
27. Logging
28. Scaling Strategy
29. Future Roadmap

30. Risks

31. Assumptions

32. Architecture Decision Summary

Cost Analysis

Provide detailed cost estimates.

Include:

Azure AI Search

Azure OpenAI

Blob Storage

Container Apps

Key Vault

Application Insights

Container Registry

Networking

Estimate costs for:

Pilot

Production

Scaling

Explain which services contribute the highest operational cost.

Diagram Requirements

Generate professional architecture diagrams using Mermaid.

Include:

High-Level Architecture

Component Diagram

Sequence Diagram

Deployment Diagram

Azure Architecture Diagram

Indexing Pipeline

Retrieval Pipeline

Coverage Analysis Flow

Data Flow

API Flow

Expectations

The document should resemble an Enterprise Architecture Design Document prepared by a Principal Solution Architect.

It should be suitable for:

- CIO Review
- Architecture Review Board
- Engineering Team
- Business Leadership
- Delivery Team

The document should be technically accurate, scalable, production-ready, cloud-native, modular, and future-proof.